apps/web-client/src/pages/ReportLost.tsx

```
import { useState } from 'react';
import { Upload, Search, MapPin, MessageSquare } from 'lucide-react';
import { uploadItem } from '../lib/api';
import { useAuth } from '../context/AuthContext';
import { ChatWindow } from '../components/ChatWindow';

export function ReportLost() {
  const [file, setFile] = useState<File | null>(null);
  const [desc, setDesc] = useState('');
  const [matches, setMatches] = useState<any[]>([]);
  const [searched, setSearched] = useState(false);
  const [loading, setLoading] = useState(false);

  // State to track which chat window is open
  const [activeChat, setActiveChat] = useState<{ itemId: string, finderId: string } | null>(null);

  // Use the global Auth Context
  const { user } = useAuth();
  const myUserId = user?.id;

  const handleFindMatches = async () => {
    if (!file) return;
    setLoading(true);
    setSearched(true);

    const formData = new FormData();
    formData.append('file', file);
    formData.append('description', desc || 'Lost item search');
    formData.append('type', 'lost');
```

```
    try {
      const { data } = await uploadItem(formData);
      setMatches(data.matches || []);
    } catch (e) {
      console.error(e);
      alert("Search failed. Please try again.");
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-4xl mx-auto mt-8 px-4 pb-20 relative">

      {/* 🟢 THE CHAT WINDOW (Renders if active) */}
      {activeChat && myUserId && (
        <ChatWindow
          itemId={activeChat.itemId}
          finderId={activeChat.finderId}
          currentUserId={myUserId}
          onClose={() => setActiveChat(null)}
        />
      )}

      {/* Hero Card */}
      <div className="bg-indigo-600 rounded-[2rem] p-8 md:p-12 text-white flex flex-col md:flex-row items-center gap-10 shadow-2xl shadow-indigo-200 overflow-hidden relative">
        <div className="absolute top-0 right-0 w-64 h-64 bg-indigo-500 rounded-full blur-3xl -mr-16 -mt-16 opacity-50"></div>

        <div className="flex-1 relative z-10">
```

```
<h1 className="text-3xl md:text-4xl font-extrabold mb-4 leading-tight">Lost something?</h1>
<p className="text-indigo-100 text-lg leading-relaxed mb-8 opacity-90">
  Upload a photo of what you lost. Our AI will scan the "Found" gallery to find visually similar items for you.
</p>

<textarea
  className="w-full mb-4 p-3 rounded-xl bg-white/10 border border-indigo-400 text-white placeholder-indigo-200 outline-none focus:bg-white/20 transition"
  placeholder="Optional description (e.g. Blue wallet)"
  value={desc}
  onChange={(e) => setDesc(e.target.value)}
/>

<div className="flex gap-4 items-center flex-wrap">
  <div className="relative w-full md:w-auto h-14 px-6 bg-indigo-500 rounded-xl border-2 border-indigo-400 border-dashed flex items-center justify-center cursor-pointer hover:bg-indigo-400 transition group">
    <input
      type="file"
      onChange={(e) => setFile(e.target.files?.[0] || null)}
      className="absolute inset-0 opacity-0 cursor-pointer"
    />
    <div className="flex items-center gap-2 text-indigo-100 group-hover:text-white font-medium">
      <Upload className="w-5 h-5" />
      <span>{file ? file.name.slice(0, 15) + '...' : 'Upload Photo'}</span>
    </div>
  </div>
  <button
    onClick={handleFindMatches}
    disabled={loading || !file}
```

```
                    className="flex-1 md:flex-none bg-white text-indigo-600 px-8 py-3.5 rounded-xl
font-bold hover:bg-indigo-50 transition disabled:opacity-50 shadow-lg"
                  >
                    {loading ? 'Scanning Gallery...' : 'Find Matches'}
                  </button>
                </div>
              </div>
            </div>

            {/* Match Results Section */}
            {searched && (
              <div className="mt-16 animate-fade-in-up">
                <div className="flex items-center gap-3 mb-8">
                  <Search className="w-6 h-6 text-slate-400" />
                  <h3 className="text-xl font-bold text-slate-800">
                    {matches.length > 0 ? `We found ${matches.length} potential matches` : "No matches
yet. We saved your request!"}
                  </h3>
                </div>

                <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
                  {matches.map(item => (
                    <div key={item.id} className="bg-white rounded-2xl p-4 shadow-sm border border-
slate-100 hover:shadow-xl hover:-translate-y-1 transition duration-300">
                      <div className="relative h-48 mb-4 overflow-hidden rounded-xl">
                        <img src={item.imageUrl} className="w-full h-full object-cover" />
                        <div className="absolute top-2 right-2 bg-green-500/90 backdrop-blur-sm text-
white text-[10px] px-2 py-1 rounded-full font-bold shadow-sm">
                          {Math.round((item.score || 0) * 100)}% MATCH
                        </div>
                      </div>
```

```
                    <h4 className="font-bold text-slate-800 text-lg mb-1">{item.description}</h4>
                    <div className="flex items-center gap-1 text-slate-400 text-xs mb-4">
                      <MapPin className="w-3 h-3" />
                      <span>Item ID: {item.id.slice(0, 8)}</span>
                    </div>

                    <button
                      className="w-full bg-slate-900 text-white py-3 rounded-xl font-bold text-sm
hover:bg-slate-800 transition shadow-lg shadow-slate-200 flex items-center justify-center gap-2"
                      onClick={() => {
                        if (item.user && item.user.id) {
                          setActiveChat({ itemId: item.id, finderId: item.user.id });
                        } else {
                          alert("Cannot chat: Finder details missing on this item.");
                        }
                      }}
                    >
                      <MessageSquare className="w-4 h-4" />
                      Chat with Finder
                    </button>
                  </div>
                ))}
              </div>
            </div>
          )}
        </div>
      );
}
```

apps/web-client/src/components/ChatWindow.tsx

```
import { useState, useEffect, useRef } from 'react';
```

```
import { useSocket } from '../context/SocketContext'; // ✅ Reuse Global Socket
import { Send, X, MessageSquare, Trash2 } from 'lucide-react';

interface ChatProps {
  itemId: string;
  finderId: string;
  currentUserId: string;
  onClose: () => void;
}

export function ChatWindow({ itemId, finderId, currentUserId, onClose }: ChatProps) {
  const { socket } = useSocket(); // ✅ Reuse connection
  const [messages, setMessages] = useState<any[]>([]);
  const [newMessage, setNewMessage] = useState('');
  const bottomRef = useRef<HTMLDivElement>(null);

  // 🔒 SECURE ID GENERATION
  // We explicitly label the Seeker vs Finder to prevent ANY collision.
  // Room ID: chat_item_ITEMID_finder_FINDERID_seeker_SEEKERID
  const conversationId = `chat_item_${itemId}_finder_${finderId}_seeker_${currentUserId}`;

  useEffect(() => {
    if (!socket) return;

    console.log("🔌 Joining Room:", conversationId);

    // 1. Join the unique room
    socket.emit('join_room', conversationId);

    // 2. Load History
    socket.emit('get_messages', conversationId, (history: any[]) => {
```

```
  console.log("📜 History Loaded:", history.length);
  setMessages(history);
});

// 3. Listen for real-time messages
const handleMessage = (msg: any) => {
  // Only add if it belongs to THIS room
  if (msg.conversationId === conversationId) {
    setMessages((prev) => [...prev, msg]);
  }
};

socket.on('receive_message', handleMessage);

// Listen for deletes
const handleDelete = (payload: { messageId: string, conversationId: string }) => {
  if (payload.conversationId === conversationId) {
    setMessages(prev => prev.map(msg =>
      msg.id === payload.messageId
        ? { ...msg, content: '🚫 This message was deleted', isDeleted: true }
        : msg
    ));
  }
};
socket.on('message_deleted', handleDelete);

// Cleanup: Leave room logic (optional) or just remove listener
return () => {
  socket.off('receive_message', handleMessage);
  socket.off('message_deleted', handleDelete);
};
```

```
  }, [socket, conversationId]);

  // Auto-scroll
  useEffect(() => {
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);

  const handleSend = () => {
    if (!newMessage.trim() || !socket) return;

    // Send to backend
    socket.emit('send_message', {
      conversationId,
      content: newMessage,
      senderId: currentUserId,
      receiverId: finderId, // Needed for notification
    });
    setNewMessage('');
  };

  return (
    <div className="fixed bottom-4 right-4 w-96 h-[500px] bg-white rounded-2xl shadow-2xl border
border-slate-200 flex flex-col z-50 animate-fade-in-up font-sans">
      {/* Header */}
      <div className="bg-indigo-600 p-4 rounded-t-2xl text-white flex justify-between items-center
shadow-md">
        <div className="flex items-center gap-2">
          <MessageSquare className="w-5 h-5" />
          <div className="flex flex-col">
            <span className="font-bold text-sm">Chat with Finder</span>
            <span className="text-[10px] opacity-75">ID: {conversationId.slice(-6)}</span>
          </div>
```

```
    </div>
    <button onClick={onClose} className="hover:bg-indigo-500 p-1 rounded-full transition">
      <X className="w-5 h-5" />
    </button>
  </div>

  {/* Messages Area */}
  <div className="flex-1 overflow-y-auto p-4 bg-slate-50 space-y-3">
    {messages.length === 0 && (
      <div className="text-center text-slate-400 text-xs mt-10">
        <p>This is the start of your conversation.</p>
        <p>Say hello! 👋</p>
      </div>
    )}

    {messages.map((msg, idx) => {
      const isMe = msg.sender?.id === currentUserId || msg.sender?.id === undefined; // Handle local echo if needed
      return (
        <div key={idx} className={`flex ${isMe ? 'justify-end' : 'justify-start'} group hover:bg-transparent`}>
          <div className={`flex items-center gap-2 max-w-[80%] ${isMe ? 'flex-row-reverse' : ''}`}>
            <div className={`p-3 rounded-2xl text-sm shadow-sm ${isMe ? 'bg-indigo-600 text-white rounded-br-none' : 'bg-white border border-slate-200 text-slate-800 rounded-bl-none'} ${msg.isDeleted ? 'italic opacity-60' : ''}`}>
              {msg.content}
            </div>
            {isMe && !msg.isDeleted && (
              <button
                onClick={() => {
                  socket?.emit('delete_message', {
                    messageId: msg.id,
```

```
                conversationId,
                senderId: currentUserId
              });
            }}
            className="opacity-0 group-hover:opacity-100 p-1 hover:bg-slate-100 rounded-full transition text-slate-400 hover:text-red-500"
            title="Delete message"
          >
            <Trash2 className="w-4 h-4" />
          </button>
        )}
      </div>
    </div>
  );
})}
<div ref={bottomRef} />
</div>

{/* Input Area */}
<div className="p-3 border-t bg-white flex gap-2">
  <input
    className="flex-1 bg-slate-100 border-none rounded-xl px-4 py-2 text-sm focus:ring-2 focus:ring-indigo-500 outline-none transition"
    placeholder="Type a message..."
    value={newMessage}
    onChange={(e) => setNewMessage(e.target.value)}
    onKeyDown={(e) => e.key === 'Enter' && handleSend()}
  />
  <button
    onClick={handleSend}
    className="bg-indigo-600 text-white p-2 rounded-xl hover:bg-indigo-700 transition shadow-md"
```

```
        >
          <Send className="w-5 h-5" />
        </button>
      </div>
    </div>
  );
}
```

apps/web-client/src/context/AuthContext.tsx

```
import { createContext, useContext, useState, useEffect } from 'react';
import type { ReactNode } from 'react';
import { checkAuth, login as apiLogin, logout as apiLogout } from '../lib/api';

interface User {
  id: string;
  email: string;
  fullName: string;
}

interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);
```

```
  // 1. Check Session on Mount
  useEffect(() => {
    checkAuth()
      .then(({ data }) => setUser(data))
      .catch(() => setUser(null))
      .finally(() => setIsLoading(false));
  }, []);

  // 2. Login Wrapper
  const login = async (email: string, pass: string) => {
    await apiLogin(email, pass);
    const { data } = await checkAuth(); // Re-fetch user data
    setUser(data);
  };

  // 3. Logout Wrapper
  const logout = async () => {
    await apiLogout();
    setUser(null);
    window.location.href = '/login';
  };

  return (
    <AuthContext.Provider value={{ user, isAuthenticated: !!user, isLoading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => {
```

```
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used within AuthProvider');
  return context;
};
```